

Problema 1: Sistema de Reservas de Espacios Co-Working (Integrador Completo)

Requisito pedagógico: Este problema abarca **todos los temas de la Unidad 2**, incluyendo la implementación de **Interfaces** y **Clases Abstractas**.

Consigna del Problema

Un moderno centro de Co-Working necesita un sistema informático para gestionar la reserva de sus diferentes espacios de trabajo.

Tras el relevamiento de requerimientos, se identificaron los siguientes puntos del dominio:

1. **Contrato de Reserva:** Todos los espacios que se puedan alquilar en el centro deben cumplir obligatoriamente con un contrato/interfaz que garantice dos capacidades:
 - Calcular el costo total de la reserva en función de una cantidad determinada de horas (`double`).
 - Consultar la disponibilidad del espacio para ser reservado (`boolean`).
2. **Jerarquía de Espacios:**
 - Debe existir una abstracción base para representarlos que prevenga la creación de "espacios genéricos" sin tipo definido. Todos los espacios poseen un identificador/código único, una capacidad máxima de personas y un precio base por hora.
 - La abstracción base debe garantizar que el identificador no sea nulo ni esté vacío, y que la capacidad y el precio base sean estrictamente mayores a cero; en caso contrario, debe lanzarse una excepción `IllegalArgumentException`.
 - En la clase base debe existir una **sobrecarga de métodos** para actualizar la capacidad del espacio: una versión sencilla que reciba la nueva capacidad, y otra versión extendida que reciba además un valor booleano para indicar si la reorganización requiere mobiliario adicional (si es verdadero, se debe añadir la leyenda " `[Mobiliario Especial]`" al identificador o descripción del espacio).
 - Se deben definir dos tipos concretos de espacios:
 - **Escritorios Individuales:** Poseen adicionalmente un atributo booleano para indicar si cuentan con monitor extra. El costo por hora es el precio base, pero si cuenta con monitor extra se adiciona un 15% al costo final. Un escritorio individual siempre está disponible para reserva.
 - **Salas de Reuniones:** Poseen un costo adicional fijo de limpieza/mantenimiento por reserva y un indicador de si incluyen proyector. El costo final se calcula como $(\text{precioBase} * \text{horas}) + \text{costoLimpieza}$. Si la sala requiere proyector pero el costo de limpieza asignado es 0, no estará disponible para reserva.

3. Organización del Código:

- Separa las clases en dos paquetes: `model` (para la jerarquía de dominio e interfaz) y `app` (para la gestión y ejecución).
- En el paquete `app`, crea una clase gestora del centro de Co-Working que administre una colección de elementos reservables mediante un `ArrayList`. Esta clase debe permitir registrar espacios, mostrar un reporte polimórfico de reservas estimadas y calcular el monto total a recaudar si todos los espacios registrados se alquilaran por una determinada cantidad de horas.
- Diseña una clase ejecutable (`Main`) donde se instancien al menos 2 escritorios y 2 salas, se prueben las sobrecargas de métodos, se capture una excepción tras un intento de inicialización inválida y se imprima el reporte final.